CSS4003 — DevOps 2

Lecture 9: AWS Storage & Databases

Nauruzbayeva Farikha · Lesson 9

Recap — Lesson 8 (AWS Networking)

- What makes a subnet "public"? (its route table sends `0.0.0.0/0` to an internet gateway — not its name, not its CIDR)
- How many addresses does a `/24` subnet have? (256, of which AWS reserves 5 — so 251 usable)
- Why does SG-app allow SSH from SG-web instead of an IP? (it follows the web servers wherever they move; an IP rule breaks when an instance is replaced)
- Security group vs NACL: which one is stateful? (the security group — return traffic is allowed automatically; NACLs need rules both ways)
- A private instance needs to download updates. What do you add? (a NAT gateway in a public subnet, and a route to it — and delete it after, it costs money)

Today’s agenda

- ☐ Block, file and object storage: three different shapes of data
- ☐ EBS: disks for EC2, volume types and snapshots
- ☐ S3: buckets, keys, storage classes, versioning and lifecycle
- ☐ Keeping S3 private: Block Public Access and presigned URLs
- ☐ S3 from the CLI — the commands you’ll use in CI/CD
- ☐ Databases: on EC2 vs managed RDS, and DynamoDB
- ☐ Choosing the right service, and not paying for forgotten ones

The scenario

Your app from last week now has users. They upload profile pictures, which the app saves to `/var/www/uploads` on the EC2 disk. The database is PostgreSQL, installed by hand on the same instance. One night the instance is terminated by mistake — and the pictures, the database and the last two weeks of data go with it.

Today: put each kind of data where it survives — and where AWS does the boring, risky parts for you.

Block 1

Three shapes of storage

Block, file, object

	Block	File	Object
Looks like	A raw disk	A shared folder	A key → a blob of bytes
You use it via	A filesystem you format	NFS mount	HTTP API (`GET` , `PUT`)
Attached to	One instance	Many instances	Nothing — reach it from anywhere
AWS service	**EBS**	**EFS**	**S3**
Good for	OS disk, databases	Shared app files	Uploads, backups, logs, static sites

The scenario's mistake: user uploads (objects) and a database (block) both lived on the instance's own disk, with nothing outside it.

Block 2

EBS — disks for EC2

EBS in four facts

- **EBS** = Elastic Block Store: network-attached disks for EC2. The root disk of your instance last week was an EBS volume.
- A volume lives in **one Availability Zone**, and attaches only to instances **in that same AZ**.
- It survives **stop/start**. On **terminate**, the root volume is deleted by default (`DeleteOnTermination`); extra volumes are kept unless set otherwise.
- You pay for the **size you provision**, attached or not — an unattached ("available") volume still costs money.

EBS volume types (qualitatively)

Type	Kind	Typical use
gp3	General purpose SSD	The default: boot disks, most apps
io2	Provisioned-IOPS SSD	Busy databases that need steady, high IOPS
st1	Throughput HDD	Big sequential reads: logs, data processing
sc1	Cold HDD	Large data you rarely touch, lowest cost

HDD types (`st1` , `sc1`) can't be boot volumes. When in doubt: **gp3**. Exact limits change — check the EBS docs before sizing a real system.

Snapshots: backups of a volume

- A **snapshot** is a point-in-time copy of an EBS volume, stored by AWS (in S3, behind the scenes — you don't see a bucket).
- Snapshots are **incremental**: after the first, only changed blocks are stored. Frequent snapshots are cheap.
- Restore = **create a new volume** from the snapshot — in **any AZ** of the region. This is how you move a disk to another AZ.
- Snapshots can be copied to another **region** for disaster recovery.

```
aws ec2 create-snapshot --volume-id vol-0abc... --description "before upgrade"
aws ec2 create-volume --snapshot-id snap-0def... --availability-zone us-east-1b
```

Instance store, and EFS

Instance store

- A disk **physically attached** to the host machine, on some instance types.
- Very fast, included in the price.
- **Gone** when the instance stops or terminates.
- Use it only for scratch data: caches, temp files.

EFS

- **Elastic File System**: a managed NFS share.
- Mounted by **many instances at once**, across AZs.
- Grows and shrinks automatically — you pay for what you store.
- Use it when several servers need the **same files**.

Quick check

Your EBS volume is in `us-east-1a`. You need the same data on an instance in `us-east-1b`. What do you do?

Take a snapshot, then create a new volume from it in `us-east-1b` and attach it there. A volume can't attach across AZs.

Block 3

S3 — object storage

Buckets and keys

- A **bucket** is a container for objects. Its name is **globally unique** across all AWS accounts — `lab09-<your-student-id>`, not `test`.
- A bucket lives in **one region**, even though the name is global.
- An **object** = a **key** (its full name) + data + metadata, e.g. `uploads/users/42/avatar.png`.
- There are no real folders: `uploads/users/42/` is just a key **prefix** that the console draws as folders.
- Objects can be from 0 bytes up to very large files; you reach them over HTTPS with the API, CLI or SDK.

Storage classes

Class	For data that is…
S3 Standard	Read often — the default
S3 Intelligent-Tiering	Accessed unpredictably — AWS moves it for you
S3 Standard-IA	Read rarely, but needed fast when read
S3 Glacier Instant / Flexible Retrieval	Archives; retrieval from instant to hours
S3 Glacier Deep Archive	Kept for years, almost never read

Colder classes cost less to store but more to read, and some have a minimum storage time. Don't put files you read every day in Glacier.

Versioning: undo for a bucket

- With **versioning on**, every overwrite keeps the old copy as a **previous version**.
- Deleting an object adds a **delete marker** instead of erasing it. Remove the marker → the object is back.
- Protects against the most common data loss: **a script or a person overwriting or deleting the wrong file**.
- Cost: every version is billed. Pair versioning with a lifecycle rule that expires old versions.

```
aws s3api put-bucket-versioning --bucket lab09-123 \
  --versioning-configuration Status=Enabled
aws s3api list-object-versions --bucket lab09-123 --prefix notes.txt
```

Lifecycle rules: automatic housekeeping

- A **lifecycle rule** moves or deletes objects by age, for a whole bucket or a key prefix.
- **Transition**: e.g. `logs/` → Standard-IA after 30 days, → Glacier after 90.
- **Expiration**: delete `tmp/` after 7 days; delete **noncurrent versions** 30 days after they're replaced.
- Set it once; S3 applies it in the background, forever.

```
{ "Rules": [{ "ID": "tidy-old-versions", "Filter": { "Prefix": "" },
  "Status": "Enabled",
  "NoncurrentVersionExpiration": { "NoncurrentDays": 30 } }] }
```

Keep it private: Block Public Access

- New buckets are **private**. Only identities you allow (IAM) can read them.
- **Block Public Access** has four switches; with all four on, no ACL or bucket policy can make the bucket public, even by mistake.
- Leaked public buckets are one of the most common cloud data breaches. Keep all four on unless you are deliberately hosting a public site.

```
aws s3api put-public-access-block --bucket lab09-123 \
  --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

Presigned URLs: share one file, briefly

- A **presigned URL** is a normal HTTPS link with a signature and an expiry time in the query string.
- Anyone holding it can `GET` (or `PUT`) **that one object** until it expires — using **your** permissions, not theirs.
- The bucket stays private. Typical use: "download your invoice" links, direct browser uploads.

```
aws s3 presign s3://lab09-123/report.pdf --expires-in 300   # 5 minutes
curl -o report.pdf "<the URL it printed>"
```

Static website hosting

- S3 can serve a **static site** (HTML, CSS, JS, images) straight from a bucket: no web server to patch.
- It needs public read access for the site files — so it's a **separate bucket**, never the one with private data.
- In production you usually put **CloudFront** (a CDN) in front, for HTTPS on your domain and caching.
- Your Slidev decks for this course could be hosted exactly this way.

S3 from the CLI — what CI/CD uses

```
aws s3 ls s3://lab09-123/                          # list a prefix
aws s3 cp build.zip s3://lab09-123/releases/v1.zip  # upload one file
aws s3 sync ./dist s3://lab09-123/site/ --delete    # mirror a folder
aws s3 cp s3://lab09-123/releases/v1.zip .          # download
aws s3api get-bucket-versioning --bucket lab09-123  # low-level API:
aws s3api list-object-versions --bucket lab09-123   #   every setting
```

- `aws s3` — high-level, file-like commands: `ls`, `cp`, `sync`, `rm`.
- `aws s3api` — one command per API call: versioning, lifecycle, policies, public-access settings.
- In week 12 your GitHub Actions pipeline will run exactly these commands to publish build artifacts.

Quick check

Versioning is on. You run `aws s3 rm s3://bucket/notes.txt`. Is the file gone?

No. S3 adds a **delete marker** as the newest version. All older versions are still there — delete the marker and the file is back.

Block 4

Databases on AWS

Option 1: install it yourself on EC2

- Full control: any engine, any version, any setting.
- But **you** do everything: OS patching, DB upgrades, backups, testing restores, replication, failover, disk monitoring.
- One instance = one AZ = a single point of failure.
- This is exactly the setup that lost data in today's scenario.

Option 2: Amazon RDS (managed)

	DB on EC2	Amazon RDS
OS and DB patching	You	AWS (in your maintenance window)
Automated backups	You script them	Built in, with point-in-time restore
Standby in another AZ	You build replication	One setting: **Multi-AZ**
Your job	Everything	Schema, data, who can connect, cost

Engines: PostgreSQL, MySQL, MariaDB, Oracle, SQL Server — plus **Aurora**, AWS's own MySQL/PostgreSQL-compatible engine.

RDS in your VPC

- An RDS instance runs **inside your VPC**, in a **DB subnet group** — a list of subnets in **at least two AZs**.
- Put it in **private subnets** with **public access: No**.
- Its security group allows the DB port (`5432` , `3306`) **only from the app servers' security group** — last week's SG-to-SG pattern.
- You connect to an **endpoint** (a DNS name), never an IP: it keeps pointing at the right server after failover or restore.

```
psql "host=lab-db.xxxx.us-east-1.rds.amazonaws.com port=5432 dbname=shop user=dbadmin"
```

Backups, Multi-AZ and read replicas

- **Automated backups**: daily snapshot + transaction logs → restore to any second within the retention period (**point-in-time restore**).
- A restore always creates a **new** DB instance with a **new endpoint**.
- **Multi-AZ**: a synchronous standby in another AZ, automatic failover, **same endpoint**. It's for availability — you can't read from it.
- **Read replica**: an asynchronous copy with its **own endpoint**, to spread **read** traffic. Can be promoted to a standalone DB.

Option 3: DynamoDB (NoSQL)

- A fully managed **key-value / document** database. No servers, no instance size, no patching.
- Every table has a **partition key** (e.g. `id`); items are looked up by key. Other attributes are **free-form** — items can differ.
- **On-demand capacity**: pay per request, no capacity planning. Good for spiky or unknown traffic.
- Choose it for simple, high-volume access by key: sessions, carts, device data, leaderboards. Choose **RDS** when you need joins, complex queries and transactions across tables.

DynamoDB from the CLI

```
aws dynamodb create-table --table-name students \
  --attribute-definitions AttributeName=id,AttributeType=S \
  --key-schema AttributeName=id,KeyType=HASH --billing-mode PAY_PER_REQUEST
aws dynamodb put-item --table-name students \
  --item '{"id":{"S":"s001"},"name":{"S":"Aigerim"},"group":{"S":"ENG-1"}}'
aws dynamodb get-item --table-name students --key '{"id":{"S":"s001"}}'
aws dynamodb scan --table-name students        # reads the whole table
```

`S` = string, `N` = number. `get-item` by key is cheap; `scan` reads every item — fine for a lab, avoid it on big tables.

Quick check

Your RDS database must keep running if one AZ fails. Multi-AZ or a read replica?

Multi-AZ: a synchronous standby with automatic failover and the same endpoint. A read replica is for scaling reads.

Block 5

Choosing, and cleaning up

Which service for which data?

Data	Service	Why
OS disk of an instance	EBS (gp3)	Block disk, survives stop/start
User uploads, backups, logs	S3 (+ lifecycle)	Durable, cheap, reachable from anywhere
Files shared by several servers	EFS	One file system, many instances
Relational app data	RDS (Multi-AZ in production)	SQL, joins, managed backups
Simple key lookups at scale	DynamoDB	Serverless, pay per request
Scratch / cache	Instance store	Fast, but lost on stop

Cost hygiene — what keeps billing

- **Unattached EBS volumes** and **old snapshots** — billed every month.
- **Versioned buckets**: old versions are billed. Add a lifecycle rule.
- **RDS instances**: billed per hour while they exist. A stopped RDS instance starts again by itself after 7 days.
- **Deleting a versioned bucket**: you must first delete **every version and delete marker**.

In Learner Lab your budget is limited. Delete what you created and click **End Lab** after every session.

Try it yourself

AWS Academy Learner Lab (`us-east-1`)

1. Bucket `lab09-<student-id>` , versioning on
2. Upload, overwrite, restore the old version
3. Delete, then undelete (delete marker)
4. Lifecycle rule + Block Public Access
5. Presigned URL with `curl`
6. DynamoDB table `students` : put, get, scan
7. Delete everything, then **End Lab**

Without AWS

The same commands against **moto**, a local AWS emulator:

```
pip install "moto[server]" awscli
moto_server -p 5003 &
export AWS_ACCESS_KEY_ID=test \
  AWS_SECRET_ACCESS_KEY=test \
  AWS_DEFAULT_REGION=us-east-1
aws --endpoint-url http://localhost:5003 \
  s3api create-bucket --bucket lab09-test
```

By the end of this lesson, you should be able to

- ☐ Explain block, file and object storage, and name the AWS service for each
- ☐ Pick an EBS volume type and use snapshots to back up or move a disk
- ☐ Create an S3 bucket with versioning, a lifecycle rule and Block Public Access
- ☐ Restore an overwritten or deleted object, and share one with a presigned URL
- ☐ Use `aws s3` and `aws s3api` from the command line
- ☐ Compare a database on EC2, RDS and DynamoDB, and choose between them

Before next lecture

- ☐ Empty and delete your practice bucket (all versions), delete the table, and **End Lab**
- ☐ Check that no EBS volumes or snapshots of yours are left
- ☐ Install **Docker Desktop** (or Docker Engine on Linux) and run `docker run hello-world`

Next week we package applications into containers — and later push their images and artifacts to the storage you used today.

Next lecture

Docker Fundamentals — images, containers and why "it works on my machine" stops being a problem.